COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	20/10/2026	**DATE SUBMITTED:**	20/10/2026
NAME:	DYLAN PAOLO M. MAGTAAN Serrano, Gabriel Immanuel R.	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 14.1 ALGORITHM COMPLEXITY

1. PROCEDURE OUTPUT

Tabulated below, are a number of functions against n (from 1 to 10)

```
A = (log2 n) {log to base 2 of n}
B = n {linear in n}
C = (* n (log2 n)) {n log n}
D = (* n n) {quadratic in n}
E = (* n n n) {cubic in n}
F = (expt 2 n) {exponential in n}
G = (expt 3 n) {exponential in n}
H = (fact n) {factorial in n}
```

n	A	B	C	D	E	F	G	H
1	0.0	1	0.0	1	1	2	3	1
2	1.0	2	2.0	4	8	4	9	2
3	1.0	3	4.0	9	27	8	27	6
4	2.0	4	8.0	16	64	16	81	24
5	2.0	5	11.0	25	125	32	243	120
6	2.0	6	15.0	36	216	64	729	720
7	2.0	7	19.0	49	343	128	2187	5040
8	3.0	8	24.0	64	512	256	6561	40320
9	3.0	9	28.0	81	729	512	19683	362880
10	3.0	10	33.0	100	1000	1024	59049	3628800

Think of this as algorithms A through H with complexities as defined above, showing growth rate versus input size n. Tabulated below are functions F, G and H from above (ie 2 to power n, 3 to power n, and n factorial). Problem size n varies from 10 to 100 in steps of 10. I have assumed that we have a machine that can perform "the principle activity of the algorithm" in a micro second (ie. if we are considering a graph colouring algorithm, it can compare the colour of two nodes in a millionth of a second). The columns give the number of years this machine would take to execute those algorithms on problems of size n (note: YEARS). This is expressed as 10^x, "10 raised to the power x".

n	F	G	H
10	10^-10	10^-8	10^-6
20	10^-7	10^-3	10^4
30	10^-4	10^0	10^18
40	10^-1	10^5	10^34
50	10^1	10^10	10^50
60	10^4	10^15	10^68
70	10^7	10^19	10^86
80	10^10	10^24	10^105
90	10^13	10^29	10^124
100	10^16	10^34	10^144

Therefore, if we have a problem of size (lets say 40) and the machine specified above, if the best algorithm is O(2**n) it will take 1 year, if the best algorithm is O(3**n) it will take 100,000 years, and if the best algorithm is O(n!) it will take

10,000,000,000,000,000,000,000,000,000,000,000 years

approximately. Out of interest, the age of the universe is estimated to be between 15 and 20 billion years old, i.e. 20,000,000,000 years. That is, even at modest values of n we are presented with problems that will never be solved. It is almost tempting to say that from a computational perspective, the universe is a small thing.

Contrast and compare two algorithms used in sorting. For the two algorithms, identify the best and worst case. Additionally, provide an analysis of the algorithms if they were to sort data with the following values of n: 100, 1,000, and 10,000 (use experimental and theoretical analysis tools used in previous activities).

	Best Case	Worst Case	Analysis
Algorithm 1 - Bubble Sort	O(N)	O(N^2)	The best case occurs when the array is already sorted, allowing the algorithm to stop after one pass. The worst case occurs when the array is in complete reverse order, which requires many comparisons and swapping involved.
Algorithm 2 - Merge Sort	O(n log n)	O(n log n)	Merge Sort best and worst case is O(n log n) because it consistently divides the array into smaller parts and merges them in a sorted order regardless of the arrangement of the elements in the array.

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

(Magtaan)

Bubble Sort & Merge Sort	Magtaan	Serrano
Best Case & Worst Case	Table 14-2 (100) Bubble Sort Merge Sort Best Case 0.0066 ms 0.051 ms Worst Case 0.0159 ms 0.0398 ms Table 14-2 (1000) Bubble Sort Merge Sort Best Case 0.3502 ms 0.3935 ms Worst Case 1.4574 ms 0.3972 ms Table 14-2 (10000) Bubble Sort Merge Sort Best Case 35.0738 ms 4.4967 ms Worst Case 139.803 ms 5.3813 ms	Table 14-2 (100) Bubble Sort Merge Sort Best Case 0.0122 ms 0.0162 ms Worst Case 0.0114 ms 0.0087 ms Table 14-2 (1000) Bubble Sort Merge Sort Best Case 0.3405 ms 0.0935 ms Worst Case 1.1415 ms 0.0924 ms Table 14-2 (10000) Bubble Sort Merge Sort Best Case 33.8643 ms 1.3046 ms Worst Case 106.179 ms 0.9591 ms

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A: Contrast and compare the two searching techniques used in class. The required deliverable for this activity would be the theoretical and empirical analysis.

For the theoretical analysis, the deliverable must be:

- Pseudocode Analysis
- Time and Space Complexities

Pseudocode Analysis

ALGORITHM 1: LINEAR SEARCH

```
LINEAR_SEARCH(arrSize, dataFind, list)
    i == 0
    IF i < arrSize DO
        IF dataFind = list[i] THEN
            RETURN
        END IF
        i++
    END IF
END
```

ALGORITHM 2: BINARY SEARCH

```
BINARY_SEARCH(arrSize, dataFind, list)
    low == 0
    up == arrSize - 1

    WHILE low ≤ up DO
        middle = FLOOR((low + up) / 2)

        IF list[middle] = dataFind THEN
            PRINT "Searching is successful"
            RETURN
        ELSE IF list[middle] < dataFind THEN
            low = middle + 1
        ELSE
            up = middle - 1
        END IF
    END WHILE
END
```

Time and Space Complexities

Complexity	Linear Search	Binary Search
Best-case time	$O(1)$	$O(1)$
Average-case time	$O(n)$	$O(\log n)$
Worst-case time	$O(n)$	$O(\log n)$
Auxiliary space	$O(1)$	$O(1)$

For the empirical analysis, the deliverable must be the filled-out table and your analysis.

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Binary Search Average Runtime: 32 ns	Linear Search Average Runtime: 541 ns
10,000	Binary Search Average Runtime: 67 ns	Linear Search Average Runtime: 11937 ns
100,000	Binary Search Average Runtime: 62 ns	Linear Search Average Runtime: 55228 ns

Provide your analysis.

Magtaan

- Linear Search traverses the data one element at a time, resulting in O(n) time complexity in the worst case. Binary Search repeatedly divides the search range in half, resulting in O(log n) time complexity when the data is sorted. Based on the experimental results, the average runtime of Linear Search increases as the input size grows, while Binary Search shows a much slower increase in runtime. These results demonstrate the difference between linear and logarithmic growth, showing that Binary Search is more efficient for larger sorted datasets.

Serrano

- Linear Search has a time complexity of O(n) because it checks elements sequentially, while Binary Search has a time complexity of O(log n) because it repeatedly divides the search range in half. Based on the result it shows that linear search takes more time as the number of elements increases, while binary search is generally faster as the array size grows because it reduces the search range with each iteration. This proves that the time complexity of the Linear and Binary Search are both true.

3. CONCLUSION

MAGTAAN

Summary of lessons learned

I learned that algorithm complexity measures the time and space an algorithm requires based on the input size. I also learned how to determine best-case, average-case, and worst-case time complexity by analyzing the number of times an algorithm performs its main operation. I learned that algorithms with exponential O(2^n) and factorial O(n!) growth become inefficient much faster as the input size increases.

Analysis of the procedure

The procedure helped me understand how theoretical time complexity can be compared with actual execution time using experimental measurements. By testing different input sizes, I observed how algorithm performance changes as the amount of data increases.

Analysis of the supplementary

The supplementary activity helped me understand the importance of choosing an appropriate algorithm. Linear Search becomes slower as the input size grows, while Binary Search is more efficient for larger sorted arrays.

How well did you think you did in this activity?

I think I performed well in this activity because I was able to understand the concepts and apply them through theoretical and experimental analysis. I also gained a better understanding of how to measure and compare algorithm performance.

What are your areas for improvement?

I need to improve my understanding of algorithm analysis and make sure that my experimental measurements are accurate and consistent. I also need more practice identifying the best, average, and worst case complexities of different algorithms.

SERRANO

Summary of lessons learned

I learned how to analyze the time complexity of algorithms and compare their theoretical and experimental performance. I also learned that different algorithms have different execution time depending on their efficiency , the size of the input and how they are made.

Analysis of the procedure

The procedure part helped me understand how to identify the best and worst case scenarios of different sorting algorithms. We tested different input sizes on bubble and merge sort, and I also learned how to measure execution time and compare the actual results with the theoretical time complexity to the experimental time complexity

Analysis of the supplementary

For the supplementary activity, I learned how Linear and Binary Search works and how their performance has a different impact based on the array size. The Linear search algorithm checks each element sequentially, while the binary search repeatedly divides the search range in half, their time complexity is O(n) and O(log n). Overall, I understand why binary search is much more efficient when it comes to a larger array size.

How well did you think you did in this activity?

I think I did well in this activity because I was able to apply theoretical and experimental analysis to sorting and searching algorithms. I also learned how to measure execution time and understand the relationship between input size and algorithm performance.

What are your areas for improvement?

I need to improve my understanding of time complexity and practice analyzing more algorithms. I also need to improve my ability to interpret experimental results and explain differences between theoretical and the actual execution times.

COMPUTER ENGINEERING

QUEZON CITY

4. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT was used to explain the concepts of the supplementary**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.